北京师范大学 地理科学学部 内部交流使用

Python 程序设计与应用

根据北京师范大学本科课程
Python 程序设计与应用
整理

整理人：王思睿

2025-2026 学年度 上

目录

1 程序设计与 Python 环境 3
1.1 程序设计与算法的概念 3
1.2 Python 语言及其特点 3
1.3 添加外部库 4
1.4 基于 IDLE 调试 Python 程序 4
2 Python 程序基础知识 4
2.1 Python 程序基本规范 4
2.2 Python 字符集和关键字 5
2.3 数据类型及表达式 5
2.4 Python 的基本语句 6
3 组合型数据及应用 8
3.1 综述 8
3.2 序列型数据的操作 9
3.3 集合和字典的操作 11
4 函数 12
4.1 函数的基本结构 12
4.2 函数间的信息传递 12
4.3 函数应用综合案例 14
4.4 排序算法 14
4.5 递归程序及应用 16
4.6 Python 中的 `main()`函数机制 16
5 面向对象程序设计的基础知识 17
5.1 面向对象程序设计简介 17
5.2 class 编程入门 17
5.3 子类与继承 19
6 Python 类库 20
6.1 Python 中的重要类库 20
6.2 在 Python 中使用外部库 20
6.3 math 库及使用 21
6.4 turtle 库及使用 21
6.5 random 库及使用 22
7 文件管理与异常读取 23
7.1 文件及其输入、输出 23
7.2 异常处理 25
8 科学计算：SciPy 和 NumPy 26
8.1 NumPy 及其运用 26
8.2 SciPy 及其运用 29
9 图像 & 图形：pillow 与 Matplotlib 30
9.1 图形、图像的概念 30
9.2 基于 pillow 的图像处理 30
9.3 基于 Matplotlib 制作统计图 31
10 文本处理与网络爬虫 32
10.1 文本处理与可视化 32
10.2 以 WordCloud 绘制词云图 33
10.3 Web 信息获取与爬虫入门 34
11 Python 环境下的统计分析 35

11.1 统计推断的概念 35
11.2 数据预处理 35
11.3 数据统计分析技术 38

第 1 章 程序设计与 Python 环境

§ 1.1 程序设计与算法的概念

1.1.1 程序设计发展历程

1. 机器语言：二进制，不用翻译解释。执行速度快。
2. 汇编语言：使用注记符号来表示机器语言，需要翻译成机器语言后才能执行。
3. 高级语言：接近自然语言，使用与自然语言语法接近的自封闭词法体系。
    - 语言处理系统：负责把高级语言的源程序转化为机器语言的软件。同一种高级语言在面对不同机型时应该使用不同的语言处理系统。
    - 高级语言的编译和解释：
        - 编译型：整体转化、集中翻译、最后执行
        - 解释型：对源程序代码一边翻译一边执行
4. 高级语言体系简介
    (1) 以早期BASIC为代表的流水账式程序：已出现基本的分支和循环结构，但尚未有函数的概念。
    (2) 以 Pascal 和 C 语言为代表的结构化程序设计：出现函数的概念——人们把能够完成一个特定功能的代码块称为函数（或过程）。
    (3) 以 C++为代表的面向对象的程序设计：出现对象的概念——对象是对即将开发的应用系统中特定物体的描述。**类是对象的模板，对象是类的实例。**

1.1.2 算法的概念

算法是与具体语言的语法结构无关的解题思路。常用表示工具有程序流程图和 N-S 流程图。

算法的五个特性：

- 有穷性：有限步骤、有限时间内结束；
- 确定性：每一步无歧义，相同输入对应相同执行逻辑；
- 可信性：所有步骤都可实际执行；
- **输入：0 个或多个合法输入；**
- **输出：至少一个结果输出。**

1.1.3 程序质量评价

- 空间复杂度：耗费计算机内存空间
- 时间复杂度：耗费计算机的运行时间

§ 1.2 Python 语言及其特点

1.2.1 Python 程序的特点

1. **面向对象、解释型、弱类型**的**跨平台**脚本语言
2. **开源型**编程语言：Python 是自由/开放源码软件（Free/Libre and Open Sourse Software, FLOSS)。
3. Python 是一种功能强大的编程语言，具有大量的外部类库
4. Python 具有极佳的可读性

1.2.2 Python 程序入门

1. Python 运行环境
    (1) 版本体系：2.x 和 3.x， 彼此不互通
    (2) 集成化编程环境：Python 自带的 IDLE
2. IDLE 中的两种运行方式
    (1) 命令行交互方式：每次输入一条语句，按回车键后该语句被直接执行。此方式直接运行在IDLE的 Shell 主窗口中。

```python
>>> 1+5 // 2
# 3
```

```
>>> print("I love you!")
# I love you!
```

(2) 程序文件方式：把多个程序语句写在一个独立程序文件中，并把程序文件命名为以.py 为拓展名的纯文本文件。

§ 1.3 添加外部库

1.3.1 直接基于 Internet 安装外部库

在命令提示符状态下输入 `pip3 install <外部库名称>`，并按回车键。

1.3.2 利用已经下载的 WHL 文档安装外部库

§ 1.4 基于 IDLE 调试 Python 程序

1.4.1 程序调试入门

程序调试是为了发现程序中的错误并及时纠正。

1. 语法错误的修正——利用系统提示修正语法错误。
2. 逻辑错误的修正——以特定值是否符合预期来检验逻辑错误。

1.4.2 利用 Python 的调试环境调试程序

Python 编程环境(IDLE)内置了程序调试功能(即 Python Debug,pdb)。

调试功能简介：

1. “Go”按钮：单机该按钮将导致程序正常执行至终止，或达到一个预设的断点。
2. “Step”按钮：逐步调试按钮——严格逐步。单击该按钮将导致调试器执行下一个代码，然后再次停下来。若遇到函数，则会步入函数内部，继续做单步调试。
3. “Over”按钮：逐步调试按钮——非严格逐步。把函数当作一条语句来执行，并不进入函数内部。
4. “Out”按钮：结束当前函数。单击该按钮，将导致调试器全速执行当前函数内的代码行，直到他从当前函数中退出——与“Step”功能相辅相成，在 Step 进入某个函数中后，可借助 Out 快速完成此函数，以便返回上一层程序。
5. “Quit”：立马终止该程序。

第 2 章 Python 程序基础知识

§ 2.1 Python 程序基本规范

2.1.1 基础语句（略）

语句 `break` 表示强制跳出循环体，语句 `continue` 表示在此趟执行中不在执行 `continue` 之后的语句，直接跳到循环开始之处。

2.1.2 常量、变量与表达式

1. 常量

   在 Python 中没有单字符的概念。Python 中的所有字符，均以字符串的形势出现。所以单引号，双引号甚至三引号都可以作为字符串的标记。

2. 变量

3. 表达式

   纯表达式不能构成独立的 Python 语句，只有把表达式赋值到变量，才能作为独立的语句使用。例如 `x = 8 + 3 - 5 * 2` 是独立的 Python 语句，但 `8 + 3 - 5 * 2` 就不是。

§ 2.2 Python 字符集和关键字

2.2.1 Python 的字符集

1. 字符集的概念：计算机系统所支持的所有字符组成的集合，称为字符集，如 ASCII，GB/T2313，Big5，Unicode。

   Unicode 采用了 16bit(2 个字节)来表示英文字符、数码符号，与 ASCII 不同。

2. Python 中采用 Unicode 编写。

2.2.2 关键字

常见的关键字(略) 数据类型标记，比如整型 `int`，浮点型 `float`，布尔型 `bool` 等也是 Python 的保留字。

2.2.3 表示符的定义及规范

1. 变量名的命名规范：
   - 数码不能在第一项
   - 不能有关键字
   - 不能有空格、运算符（`+`,`-`,`*`,`/`,`%`,`!`,`^`等）均不可作为组成部分。（可以有 `$` 字符）
   - 一些其它语言常见的数据类型如 int、float、double，在 Python 中仍为一种类型，通常作为程序的保留字，不作为标识符。
2. 其他约定：
   - 变量名、方法名均使用小写字母开头，中间可以包含大写字母或数码。
   - 类名称的首字母大写
   - 外部库（包）名称全部小写
   - 文字和常数的标识符全部大写

§ 2.3 数据类型及表达式

2.3.1 数据类型简介

1. 基本数据类型：整型、浮点型、逻辑型等

   仅需申请存储其值的内存空间，不包含针对这类数据实施操作的方法体（程序代码），因此占用的空间很少。例如 `int` 变量只占据 4 个字节的内存空间。

   (1) 整型
   - `int` 默认占用 32bit(4 个字节)，可表示的范围为$-2^{31} \sim 2^{31}-1$
   - `long` 默认占用 64bit(8 个字节)，可表示的范围为$-2^{61} \sim 2^{61}-1$

     3.x 版本中被合并为 `int`

   (2) 浮点型
   - 表示方法：纯小数$X * 2^{Y}$ 其中阶码Y表示小数点的位置
   - `float` 占用 64bit，可表示的范围为$-10^{308} \sim 10^{308}$
   - 可用科学计数法表述，如：`12.3E28` 或 `12.3e28` 表示12.3×10^{28}

   (3) 布尔型
2. 组合数据类型：字符串，列表，数组。

2.3.2 数据类型转换

1. 字符串类型转化为数值类型
2. 数值类型转化为字符串类型

   利用字符串格式符 `format` 把数值类型的量嵌入字符串内，以实现针对数值类型的类型转化。其格式为`"字符串名".format(参数表)`。例如`"{:5d}".format(x1)`其中`{}`代表字符串中内含的参数，它会以此用 `format` 内包含的变量值代替。`":"`之后的 `5d` 表示变量 `x1` 的输出格式为 5 位整数。若是浮点数，则可用 `8.3f` 的格式，其中 8 代表总宽度，3 代表小数位数。
3. 重要函数 `eval()`的使用

   对字符串形式的表达式求值，如果该表达式中只包含一个数字串，则会实现类型转化的功能。

```python
xx = eval("35 * 25 + 61")
print(xx)
xx = "你好,我35岁了" # 字符串中各字符的索引（从0开始）
yy = eval(xx[4:6]) #对xx进行切片操作后，用eval()执行并将结果赋值给yy
print(yy)
```

2.3.3 重要运算符

1. 赋值运算符：允许连续赋值，从右至左进行
2. 算术运算符：在运算过程中，简单数据类型可以自动地转化为复杂的数据类型（即占用字节数较少的数据类型可以自动转化为占用字节数较多的数据类型）。
3. 逻辑运算符
4. 位运算符
   (1) 位的“与”“或”“非”“异或”运算：这类运算符对两个整数的对应二进制位逐一进行逻辑运算，返回结果为十进制整数。

   `&`：按位与，两个位都为 1 时，结果位才为 1，否则为 0

   `|`：按位或，两个位只要有一个为 1，结果位就为 1，否则为 0

   `~`：按位非，对单个整数的所有二进制位取反（0 变 1，1 变 0）

   `^`：按位异或，二者不相同时结果为 1

   (2) 移位运算符

   `<<`：向左移动若干位，例如：`4<<3` 代表把数据 4 向左移动 3 位，结果为 32.对于一个较小的正整数，每左移一位，相当于数值乘 2。

   `>>`：向左移动若干位，例如：`4>>3` 代表把数据 4 向右移动 3 位，结果为 1.对于一个较大的正整数，每右移一位，相当于数值除以 2。

优先级：算术>比较>逻辑（非>与>或）

§ 2.4 Python 的基本语句

2.4.1 输入语句

`input()`获得的信息一定是一个字符串，如果获得的字符串不是纯粹的数字串，则可以借助字符串的`find()`方法提取所需的字符串。

2.4.2 输出语句

1. 带有格式的输出——基于 %

```python
xx = 98
yy = 93.7
print("xx的值是%5d，yy的值是%8.2f"%(xx,yy))
#以5位整型方式输出xx，宽8位、2位小数方式输出yy
```

`d` 表示整数，`f` 表示浮点数，`s` 表示字符串，`%` 表示参数，参数必须用括号括起来。

2. 带有格式的输出——基于{}

f-string 的基本语法非常简单，只需在字符串前加上一个小写的 `f` 或大写的 `F`，然后在字符串内部使用花括号 `{}`来包含变量或表达式。例如： 冒号前为变量名，冒号后为格式符。

```python
xx = 98
yy = 93.7
print(f"xx的值为{xx:5d},yy的值为{yy:8.2f}")
```

3. 带有格式的输出——基于 format

```python
xx = 98
yy = 93.7
```

```
3 print("xx的值为{:5d}，yy的值为{:8.2f}".format(xx,yy))
4 print("xx的值为{0:5d}，yy的值为{1:8.2f}".format(xx,yy))
```

4. 换行问题

默认输出后换行，end=""表示不换行，输出空字符。

2.4.3 选择结构（略）

2.4.4 循环结构

1. while 循环结构

```python
while (循环条件):
  循环体
```

- 案例：输出 1~100 之和

```python
Sum = 0
i = 1
while (i<=100):
  Sum = Sum + i
  i += 1
print(Sum)
```

- 条件永真的设置：

```python
Sum = 0
i = 1
while True:
  Sum = Sum + i
  i += 1
  if i > 100:
    break
print(Sum)
```

2. for 循环结构

```python
for <变量名> in <集合>:
  语句块
```

- 常用：

```python
for i in range(n) #自动执行n次
for i in range(n1,n2)
for i in range(n1,n2,step)
```

案例：输出 1~100 之和

```python
sum = 0
for i in range(1,101):
  sum = sum + i
print(sum)
```

第 3 章 组合型数据及应用

§ 3.1 综述

3.1.1 Python 内置的组合型数据

把对象的值与针对对象值的各种操作封装在一起，构成相对独立的 class 块，是面向对象程序设计的主要思想。Python 是典型的面向对象的程序，因此 Python 中的每一种组合型数据类型，都是“变量的值+相关操作方法”的组合体。

1. 序列类型：有下标，可以利用下标调用单个元素；带有多个方法可以实施，如 `append`、`len` 等。
   - 字符串
   - 元组
   - 列表
2. 非序列类型
   - 集合：`set` 类型，集合无序，不可重复。
   - 字典：映射关系，特点：“键名：键值”组成的数据对。

3.1.2 序列类型及其特点

1. 字符串

   在计算机的内存中，字符串采用的是一种**静态存储结构**。因此，在程序设计中，若字符串的内容被修改，通常需要字符串变量整体地放弃原来的存储空间，然后重新申请新的内存空间。*所以，通常不建议直接对大字符串执行高循环度的修改操作。*

   因此，字符串被认为是“不可原地修改”的类型，**即一旦完成定义，就不能在原内存位置修改其内容。**

2. 元组

   在计算机的内存中，元组采用的是一种**静态存储结构**。因此，在程序设计中，若元组的内容被修改，通常需要元组变量整体地放弃原来的存储空间，然后重新申请新的内存空间。*所以，不允许直接修改元素的值。*

   因此，元组被认为是“不可原地修改”的类型，**即一旦完成定义，就不能在原内存位置修改其内容。**

3. 列表

   列表采用**动态存储结构**，由于元素之间的存储采用的指针方式（链式结构），所以**允许对列表的任意元素做修改操作。**

3.1.3 集合与字典类型及特点

在 Python 中，集合和字典均有“{}”作为标记，其中的元素没有顺序关系，而且绝对不允许重复。

1. 集合

```python
x1 = {1,2,4,"计算机教学",(12,23),2,1}
# 这个式子定义了一个集合，并赋值给新变量x1，但集合值为{1,2,4,"计算机教学",(12,23)}，因为*重复元素被自动剔除了。*
```

2. 字典：可以通过键名从字典中提取出特定的数据。字典类型也被称为“映射类型”。
3. 小结
   (1) 字典和集合均为无序的元素集类型。因此，在这两种类型中，均没有下标的概念，不可以通过下标修改或提取其中的元素
   (2) list 型变量和常量均不可以作为集合（字典）的元素
   (3) 借助于 `set()`函数，可以把其他的组合类型强制转化为集合类型。在执行集合类型转化时，会自动对其他类型内的动态元素强制转化，并删除重复的元素。例如：

```python
#字符串会被拆分为单个字符作为集合元素，再去重（空格、符号也会被当作字符）
```

```
my_str = "hello world"
my_set = set(my_str)
print(my_set)  # 输出：{'h', 'e', 'l', 'o', ' ', 'w', 'r', 'd'}（去重了重复的 'l'）
#关键特性：字典转化为集合时，仅取字典的“键（key）”，忽略值（value），且自动去重（字典本身键不重复，所以主要是格式转化）。
my_dict = {"name": "张三", "age": 18, "age": 20}  # 字典中重复的键会被覆盖（实际存储：{"name": "张三", "age": 20}）
my_set = set(my_dict)
print(my_set)  # 输出：{'name', 'age'}（仅取键）
```

§ 3.2 序列型数据的操作

3.2.1 字符串

1. 求长度

函数 len()被用于求取字符串的长度，其语法格式为“len(字符串名称)”。不是方法体形式的函数，与后续函数的使用有很大不同。

2. 切片

```python
book = 'Python 程序设计'
dd = book[2]  #结果为't'
dd = book[2:] #从第3个到最后一个，结果为'thon程序设计'
dd = book[:2] #取两个，结果为'Py'
dd = book[-2:]#结果为'设计'
dd = book[:-2]# 去掉最后两个，答案为'Python 程序'
#若 book 长度小于 2，切片不会报错，会返回原对象的全部元素（生成新对象）
```

3. 常用方法

用法：yy = str.方法名()

```python
strip()              #删除字符串两边的空白符（包括'\n','\t','\r'）
capitalize()         #首字母大写
upper()              #所有字母大写
lower()              #所有字母小写
find("内容")         #查找位置，找不到返回-1
startswith("内容")   #判断是否为指定开头
replace("old","new")  #替换
"标记符".join(复合变量)#用指定的标记符把复合变量内的各个元素拼接起来，形成新的字符串
```

4. format 用法：

(1) 位置匹配（按参数顺序）

```python
# 示例1：多个位置参数
name = "小明"
age = 20
print("我叫{}，今年{}岁。".format(name, age))
# 示例2：重复使用同一个参数（通过索引指定）
print("A={0}, B={1}, 再次A={0}".format(10, 20))
```

(2) 关键字匹配

```python
print("姓名：{name}，身高：{height}cm".format(name="小红", height=165))
```

(3) 直接传入字典（用*解包）

```python
person = {"name": "老王", "age": 35, "city": "北京"}
print("姓名：{name}，年龄：{age}，城市：{city}".format(**person))
```

(4) {参数：格式说明符}控制数值、字符串、日期等的输出格式（如保留小数、进制转换、对齐等）。

5. 转义字符：实际上为一个字符。

```
- \n 换行符
- \r 回车符
- \t 水平制表符
- \b 退格符
- \\ 代表符号\
- \" 代表符号"
- \' 代表符号'
```

3.2.2 列表与元组的操作方法

1. 列表的定义与操作

(1) 定义列表：可直接定义，也可：

```python
xx = [初值 for x in range(元素个数)]
xx = []
xx += "1234"
print(xx)  # ['1', '2', '3', '4']
xx += range(3)
print(xx)  # ['1', '2', '3', '4', 0, 1, 2]
xx += map(str,range(3))
print(xx)  # ['1', '2', '3', '4', 0, 1, 2, '0', '1', '2']
```

(2) 提取列表内容

```python
yy = xx[2]#提取下标为2的元素
zz = xx[2:4]#提取下标为2、3的元素
xx[2] = "LOVE" #针对下标为2的元素赋新值
```

(3) 为列表新增元素

- 将数值 9 添加到列表 xx 的末尾：`xx.append(9)`
- 把字符“T”插入到列表 xx 的末尾：`xx.insert(2,'T')`
- 使用列表对象的 `extend()`方法可以将另一个可迭代对象的所有**元素**添加至该列表对象尾部，**而不改变内存地址**。

  注意：

```python
nums1 = [1,2,3]
nums1.append([5,6])  # 把[5,6]当作1个元素加进去
print(nums1)  # [1, 2, 3, [5, 6]]

nums2 = [1,2,3]
nums2.extend([5,6])  # 把5、6分别当作元素加进去
print(nums2)  # [1, 2, 3, 5, 6]
```

(4) 其他

```python
remove() #删除指定值
del() #删除指定位置
pop() #删除并获取被删除的元素
sort() #升序
sort(reverse=True) #降序
reverse() #逆序
```

注：`sorted` 适用于任何可迭代对象，且是非原地修改。

2. 元组（略）

§ 3.3 集合和字典的操作

3.3.1 集合

1. **定义集合时不能将动态元素作为集合的元素，否则会报错。**只能是整型、浮点数、字符串、元组。系统会自动删除重复的元素。
2. 添加与删除元素

```python
cc.add(12)
cc.discard(12)
```

3. 集合的运算
   - x1&x2 作为交集
   - x1|x2 作为并集
   - x1-x2 作为差集

3.3.2 字典

1. 列表等动态元素不可作为元素。不允许出现重复的元素。
2. 整型量、浮点型量、字符串和元组量均可作为键名使用，但动态数据（如列表）不可以作为字典中的键名使用。
3. 把字典转化为集合后，集合内仅保留键名项，键值信息被抛弃。
4. 提取信息：<字典名>[键名项]
5. 为字典添加新元素

```python
xx["语文分"] = 92
xx.update({"语文分": 92})
```

6. 把字典中某元素删除

```python
#clear清空字典所有键值对（保留字典对象）
d = {"name": "Tom", "age": 18, "gender": "male"}
d.clear()
print(d)  # {}（字典对象还在，内容清空）
print(type(d))  # <class 'dict'>（仍为字典类型）

#pop 删除指定键的键值对（返回对应值）
d = {"name": "Tom", "age": 18, "gender": "male"}
# 1. 删除存在的键，返回对应值
age = d.pop("age")
print(age)  # 18（返回被删的值）
print(d)    # {'name': 'Tom', 'gender': 'male'}（原字典已修改）
# 2. 删除不存在的键，指定默认值（不报错）
score = d.pop("score", 0)
print(score)  # 0（返回默认值）
print(d)      # {'name': 'Tom', 'gender': 'male'}（字典不变）
#3. 删除不存在的键，未指定默认值（报错）
d.pop("score")  # KeyError: 'score'

#del 删除指定键的键值对 / 删除整个字典对象
d = {"name": "Tom", "age": 18, "gender": "male"}
# 1. 删除指定键的键值对
del d["gender"]
print(d)  # {'name': 'Tom', 'age': 18}（原字典已修改）
# 2. 删除不存在的键（报错）
del d["score"]  # KeyError: 'score'
```

```
# 3. 删除整个字典对象
del d
print(d)  # NameError: name 'd' is not defined（字典对象已被删除）
```

7. 提取键值：字典名[键名]，如：yy[“姓名”]
8. 显示所有键名：keys()

第 4 章 函数

§ 4.1 函数的基本结构

4.1.1 函数的构件及组织

1. 函数名及形参说明

```
def<函数名>([参数表]):
    [实现函数功能的语句块]
```

在函数定义时，位于函数名括号后边，括号内的变量名被称为形参，在之后调用函数时所带的参数，被称为实参。它必须时已经定义且被赋值的，即带有真实数据或内存地址的参数。二者的名称可以不相同，但类型、个数必须相同。

2. 函数的返回值

若函数被定义为无返回值函数，则在调用时直接书写函数名称并给予实参即可。例如 inputData(xx)。

若带返回值参数，则在调用此函数时应该使用赋值语句，或者直接在输出语句中直接输出返回值。例如 xx = input()。

§ 4.2 函数间的信息传递

4.2.1 变量的作用域

1. 变量的类别
   (1) 全局变量

   主程序，顶格定义。全局变量的作用域是整个程序，但不会自动作用于程序内部。**当局部作用域中出现同名的局部变量时，全局变量会被「屏蔽」，程序会优先使用局部变量。**

   (2) 局部变量

   在函数内部定义的变量，即使与全局变量同名，在不加 global 的条件下，也默认为局部变量。

   某变量位于函数内部但在函数外有同名变量，且未标记 global，简单变量可向内传值但不回传，组合变量双向传值。

2. 基于参数的函数间信息传递
   (1) 主程序对子程序的正向信息传递
   (2) 子程序对主程序的反向信息反馈

   ① 值传递规则

   对于**基本数据类型**的实参，在主程序调用函数时，调用语句中的实参被取值后传递到函数内部，由函数内部的新变量接收其值并参与各种运算。**因此，与形参变量相关的各类更新将不会回传主程序。这种信息传递方式被称为“值传递”。**

   ② 引用传递规则

   对于组合型数据的形参（如字符串、列表、类变量等），由于数据结构比较复杂，在调用函数时通常不是传值，而是直接把实参的内存地址（或实参的 ID 标记）传递到函数内部。**因此，与形参变量相关的各类更新将会回传主程序。这种信息传递方式被称为“引用传递”。**

3. 基于返回值的反向信息传递
4. 以 global 声明全局变量实现信息互传

示例：

```python
# 全局简单变量（int 不可变）
global_var = 10
def func():
    # 1. 向内传值：读取全局变量的值（此时函数内未定义同名局部变量，遵循 LEGB 找全局）
    print("函数内读取全局值：", global_var)  # 输出：10
    # 2. 尝试修改：此时 Python 认为是定义新的局部变量（覆盖全局同名变量）
    global_var = 20  # 这里的 global_var 是局部变量，和全局无关
    print("函数内修改局部值：", global_var)  # 输出：20
func()
# 外部查看全局变量：未被修改（不回传）
print("函数外全局值：", global_var)  # 输出：10
```

```python
# 全局组合变量（list 可变）
global_list = [1, 2, 3]
def func():
    # 1. 向内传值：读取全局变量的内容（容器地址不变，直接访问）
    print("函数内读取全局值：", global_list)  # 输出：[1,2,3]
    # 2. 修改内容：直接操作容器内的元素（不改变容器的地址）
    global_list.append(4)  # 不是赋值给 global_list，而是调用容器的方法修改内部
    print("函数内修改后：", global_list)  # 输出：[1,2,3,4]
func()
# 外部查看全局变量：内容被修改（回传成立）
print("函数外全局值：", global_list)  # 输出：[1,2,3,4]
```

5. 可变参数：fun(*a)
   (1) *args：这种参数允许函数接受任意数量的位置参数。在函数定义中，*args 表示一个元组，其中包含了所有传递给函数的位置参数；
   (2) **kwargs：这种参数允许函数接受任意数量的关键字参数。在函数定义中，**kwargs 表示一个字典，其中包含了所有传递给函数的关键字参数。
6. 匿名函数 lambda：是指一类无需定义标识符（函数名）的函数或子程序。所谓匿名函数，通俗地说就是没有名字的函数，lambda 函数没有名字，是一种简单的、在同一行中定义函数的方法。

```python
lambda 参数：表达式
比如：
a = lambda x:x*x
print(a(3))
举例：
#筛选列表中符合条件的数据
my_list = [2,3,4,5,6,7,8]
new_list = list(filter(lambda a:(a / 3 >= 2),my_list))
print(new_list)
#根据字符串长度给列表排序
lst = ['one', 'two', 'three', 'four', 'five', 'six']
l1 = sorted(lst, key=lambda a:len(a) )
print(l1)
```

§ 4.3 函数应用综合案例

4.3.1 求阶乘

```python
def jiecheng(n):
    jiech = 1
    for i in range (1,n+1):
        jiech *= i
    return jiech

n = int(input("请输入整数"))
print("{0}的阶乘是{1}".format(n,jiecheng(n)))
sum = 0
for i in range(1,n+1):
    sum = sum + jiecheng(i)
print(sum)
```

4.3.2 求质数

```python
def zhishu(n):
    m = n//2
    k = 2
    while k<=m:
        if n%k == 0:
            break
        k = k+1
    if k<=m:
        return False
    else:
        return True
n = int(input("请输入正整数: "))
sumx= 0
for i in range(2,n+1):
    if zhishu(i):
        print(i)
        sumx += i
print('2到{0}之间所有质数之和为{1}'.format(n,sumx))
```

§ 4.4 排序算法

4.4.1 选择排序

扫一遍索引之后的整个序列，找到最小值，把它移到序列最左边。

```python
def sortSelect(dat):
    alen = len(dat)
    for i in range(alen - 1): # 外层循环：控制选择排序的轮数，循环到倒数第二个元素即可（最后1个元素无需比较）
        temp = i  # 初始化最小值的索引为当前轮次的起始索引i
        for j in range (i+1,alen): # 内层循环：从i的下一个元素开始，遍历到列表末尾，寻找最小值的索引
            if dat[j] < dat[temp]:
                temp = j # 更新最小值的索引为当前j的位置
        dat[i],dat[temp] = dat[temp],dat[i]
        print(dat)
sortSelect(dat)
print(dat)
```

4.4.2 未优化的冒泡排序

从左到右扫描排序序列，若发现左侧的数据比右侧的大，则二者交换。每一趟排序都需要 n-1 次比较，每一趟都能把一个最大值移到右侧。共需 n-1 趟完成这个排序。

```python
dat = [1,3,453,34,224,63,23]
def sortBubble(dat):
    # 计算需要比较的最大下标：列表最后一个元素下标是 len(dat)-1，赋值给alen
    alen = len(dat) - 1
    # 外层循环：控制冒泡排序的轮数，最多需要循环 alen 轮
    for i in range(alen):
        # 内层循环：从列表头部开始，相邻元素两两比较
        for j in range(alen):
            # 判断：如果前一个元素 > 后一个元素，说明顺序颠倒
            if dat[j] > dat[j+1]:
                # 执行元素交换，把大的元素向后移
                dat[j], dat[j+1] = dat[j+1], dat[j]
sortBubble(dat)
print(dat)
```

4.4.3 优化后的冒泡排序

如果在某一趟比较中，没有发生任何交换，即可停止比较。对冒泡排序的优化包括两点：一、减少每趟待比较的数据；二、设置交换标记，若某一趟未发生交换，就立即停止交换过程。

```python
dat = [1,3,453,34,224,63,23]
def sortBubble(dat):
    alen = len(dat) - 1
    for i in range(alen):
        count = 0
        for j in range(alen-i): #此处优化，减少i次比较
            if dat[j] > dat[j+1]:
                dat[j],dat[j+1] = dat[j+1],dat[j]
                count += 1 #若有交换，则计数加1
        if count == 0: #若本趟次数为0，则结束
            break

sortBubble(dat)
print(dat)
```

4.4.4 插入排序

把 w 之处的数据插入 0 (w-1)的思路：首先提取 w 处的数据并存入 temp 变量：其次以 w-1 处为起点，若该处的数据>temp，则该数据向后移动一个位置；借着向前测试，重复上述操作，直到当前位置的数据比 temp 小或者到达左边界为止。此时当前点后面的空位就是 temp 所在的位置，直接把 temp 插入到当前点之后的空位。

```python
dat = [1,3,453,34,224,63,23]
def sortInsert(dat):
    alen = len(dat)
    for i in range(alen): #把i处的数据插入到0~(i-1)的序列中
        temp = dat[i]
        j = i - 1
        while(j>=0 and dat[j]>temp): #寻找能够存放新数据的空位
            dat[j+1] = dat[j]
            j -= 1
```

```
        dat[j+1] = temp #把待插数据放到空位上
sortInsert(dat)
print(dat)
```

4.4.5 归并排序

同时对两个列表从左向右扫描，若发现哪个列表的当前值较小，就把该值从原列表复制到目标列表。若某个列表已经结束，则把另一个列表内剩余的所有数据项复制到目标列表。若某个列表已经结束，则把另一个列表内剩余的所有数据复制到该目标列表的末尾。

```python
dat1 = [1, 3, 23, 34, 63, 224, 453]
dat2 = [2,4,5,24,35,67,345,980]
def sortComb(dat1,dat2):
    alen1,alen2=len(dat1),len(dat2)
    i,j = 0,0
    dat = []
    while (i<alen1 and j<alen2): #从两个列表同步检查，找小的
        if dat1[i] == dat2[j]:
            dat.append(dat1[i])
            dat.append(dat2[j])
            i += 1
            j += 1
        elif dat1[i] < dat2[j]:
            dat.append(dat1[i])
            i += 1
        else:
            dat.append(dat2[j])
            j += 1
    while (i<alen1): #若dat1有剩余，把剩余附加到dat
        dat.append(dat1[i])
        i += 1
    while (j<alen2):
        dat.append(dat2[j])
        j += 1
    return dat
dat = sortComb(dat1,dat2)
print(dat)
```

§ 4.5 递归程序及应用

递归不能使得计算机的计算更简单，运算量更少。相反，与人工绞尽脑汁直接设计出非递归程序相比，递归程序需要耗费更多的计算机资源，在其运算过程中，需要更多的堆栈、环境变量保存，因此其工作效率通常远远低于非递归程序。

§ 4.6 Python 中的 main()函数机制

囿于传统的 C 语言和 Java 语言体系，很多程序员热衷于设计以函数的方法管理所有的程序代码。因此，可以用 main()统领所有的主程序代码，在所有的函数代码之外，仅留一条调用函数 main()的语句。

第 5 章 面向对象程序设计的基础知识

§ 5.1 面向对象程序设计简介

5.1.1 理论基础

1. 对象与类

(1) 对象

对于客观存在的对象，通常包含两个方面的信息：静态属性和动态操作。

- 静态属性：是指一个对象所具备的属性**及其取值**。例如：学生“小明”，性别是“女”，年龄是“11”。
- 动态操作：指与此对象相关的操作。例如：对“小明”，计算她的平均分就属于与其相关的操作。

在面向对象的程序设计中，对象的静态属性叫“成员变量”，而反映动态操作的函数叫做“方法”或“方法体”。

(2) 类 class

类是对同一种类对象的描述，是对象的模板。基于“类”，可以派生出许多个对象。所谓面向对象的程序设计，就是设计好类，然后利用类创造出新对象，并通过类之间的协同，实现特定功能的过程。

5.1.2 面向对象程序设计的基本特征

面向对象程序设计主要有一下特征：抽象、封装、继承、多态。后三者为三大基本特征。

1. 抽象
   - 数据抽象（属性抽象）—— 抽取「核心属性」，隐藏「无关属性」
   - 行为抽象（方法抽象）—— 抽取「核心行为」，隐藏「实现细节」
2. 封装

通过封装，可隐藏对象的属性和实现细节，仅对外部公开的特定接口，以限制外访问。

目的：增强安全性；避免不同对象的同名变量引起冲突，避免外部程序对对象内部隐藏数据的越界访问；简化编程。

3. 继承

创建新的类基于另一个已存在的类，从而提升程序的开发效率。继承使子类自动具有父类的各种属性和方法，还可以重新定义某些属性、重写某些方法。

Python 可以多重继承。

4. 多态性

当用户针对父类下达某一操作指令（虚方法）时，系统就会要求隶属于子类的所有对象自动选用适合自己的操作（实方法）。

§ 5.2 class 编程入门

5.2.1 自定义 class

1. class 的构件

(1) 成员变量/属性

① 在以 class 每创建一个对象时，都会为此对象创建一套成员变量的副本。成员变量是附着于对象的。

② 变量可声明为公开类型或私有类型。公开的成员变量可以以对象名.成员变量名的形式在 class 外使用。而私有的成员变量则不可以在 class 之外直接访问。

③ 若成员变量之前带有两个连续的下划线，就会被认为是私有类型的。

(2) 成员函数/方法：主要是面向他所隶属的对象。

(3) 类变量：只属于类自身的变量，直接以类名.变量名的形式来获取类变量的值。

(4) 类函数

(5) 构造函数

构造函数是 class 中用于初始化成员变量的一个函数，这是一个私有的、内置的函数，函数名以__init__标记，后面为形参表。形参表中的第一个参数一定是 self，用于表示当前对象，然后才是其他形参变量。

2. class 的固定格式

(1) 定义 class

```python
class<类名称>():
  <类变量名> = <默认值> #定义类变量
  ......
  def __init__(self,形参表):
    self.<成员变量名>=<形参值>
    ......
  def <成员函数名>(self,形参表):
    ......
```

注意：所有用到成员变量的地方，必须加注 self.标记

(2) 示例

```python
class Student(): #定义学生类
  count = 0 #定义类变量
  def __init__(self,xno,xname,score1,score2): #构造函数
      self.xno = xno #定义成员变量
      self.xname = xname
      self.score1 = score1
      self.score2 = score2
      self.__sjl="" #定义私有变量
  def setjl(self,sjl): #定义成员函数，为简介赋值
      self.__sjl = sjl
  def getjl(self): #定义成员函数，获取简介值
      return self.__sjl
  def getAvg(self): #定义成员函数，获取成绩均值。
      return (self.score1 + self.score2)/2
```

5.2.2 创建对象并操作对象

1. 利用 class 创建对象

新建对象的命令“对象名 = 类名(实参表)”，其中，参数表的个数、数据类型应与构造函数的格式完全一致。

2. 示例

```python
def main():
  stu1 = Student(13,"SunPing",99,87) #定义3个学生对象
  stu2 = Student(243,"WangHong",65,89)
  stu3 = Student(34,"LiuHua",87,97)
  stu1.setjl("优秀") #为私有成员变量赋值，由于sjl为私有成员变量，不允许在class之外直接访问，因此专门设置了两个成员函数对私有变量sjl实现操作。
  stu2.setjl("良好")
  print("一同学：",stu1.xno,stu1.xname,stu1.score1,stu1.score2,stu1.getAvg())
  #对象内的公共成员变量和公共成员函数均可通过“对象名.变量名”和“对象名.函数名（实参表）的形式访问。
  print("二同学：",stu2.xno,stu2.xname,stu2.score1,stu2.score2,stu2.getjl())
  print("三同学：",stu3.xno,stu3.xname,stu3.score1,stu3.score2,stu3.getjl())
```

```
main()
```

注意：对于类变量在主程序中可以利用 `Student.count` 对其赋值，或直接打印其值。

若不存在与类变量同名的成员，那么以.类变量名的方式获取类变量的值，也是可以的。但是，切不要以“.类变量名 = 值”的方式对类变量赋值，此方式会导致成员变量和类变量的结构混乱。

§ 5.3 子类与继承

5.3.1 类的继承

在定义 class 时，若未声明具体继承哪个类，则默认继承 Object 类。

Python 对继承的规定：

```python
class <子类名称>(<父类名>):
  <子类的类变量>
  <子类的构造函数>
  <子类的成员函数>
```

注意：

- 子类不能继承父类内部被标记为“私有”的成员变量和成员函数，另外，子类的独有成员变量在子类的构造函数中定义。
- 如果在子类中创建了与父类中某个函数完全相同的函数（函数名和参数表均相同），则对于子类对象而言，子类中的新函数已经覆盖了父类中的原函数。在程序运行的过程中，如果需要调用这个函数，则父类对象与子类对象各自使用自己定义的函数代码。
- 在子类初始化时，如果需要借助父类的构造方法实现，则必须显式地调用父类的构造方法“`__init()__`”。因为在进行类型实例化（创建对象时），子类的构造方法不会自动调用父类的构造方法，如果在子类初始化时找不到对父类的调用，则会报严重错误。
- 如果需要在子类中调用父类的方法，通常会以“父类名.方法名(self,其他形参)”或“super().方法名(形参表)”的方式调用。在调用时，务必正确地使用 self 参数项。

案例：

```python
class Person(): #定义人的类，括号可以省略
    def __init__(self,Id,sname,birth): #构造函数，class中的形参表第一个必定是self
        self.Id = Id
        self.Name = sname
        self.birth = birth
    # 给当前对象绑定属性，把构造函数接收到的形参值，赋值给对象自己的属性
    def getInfo(self): #在Person类中，定义一个普通方法，输出人员的个人信息，该方法只有一个形参，因为这个方法只需要调用“当前对象自己的属性”，不需要额外传其他参数。
        strx = "编号：" + str(self.Id) + ";姓名：" + self.Name #使用“+”拼接字符串
        strx = strx + ";生日：" + self.birth
        return strx
class Student(Person): #定义学生类，从Person继承
    def __init__ (self,Id,sname,birth,score1,score2,school):
        Person.__init__(self,Id,sname,birth) #调用父类的初始化函数，必带self
        self.score1 = score1
        self.score2 = score2
        self.School = school
    def getAvg(self):
        pp = (self.score1 + self.score2)/2
        return pp
    def getInfo(self): #方法重写 override 调用父类对象的getInfo()，执行父类的代码，调用子类对象的getInfo()执行子类的代码。
        strx = super().getInfo() + ";\n" #调用父类的getInfo函数
```

```
22          strx += '学校:'+self.School + ";平均成绩: " + str(self.getAvg())
23          return strx
24  def main():
25          print('请输入学生信息')
26          xh = int(input("学号: "));xm = input("姓名: ")
27          bth = input("生日: ")
28          p = Person(xh,xm,bth) #实例化对象，根据Person类的模板，创建一个具体的“人”的对象，并把这个对象绑定给变量p。括号里的xh，xm，bth是用户传入的参数，会传给Person的构造函数__init__，自动给这个对象赋值
29          print(p.getInfo())#调用p这个对象的getInfo()方法，拿到返回信息的字符串，然后打印出来。
30          ss = Student(1,'bbb','2005-11-12',28,98,'81school') #学生对象的创建
31          print(ss.getInfo())
32
33  main()
```

第 6 章 Python 类库

§ 6.1 Python 中的重要类库

1. math：标准库，无需单独安装
2. turtle：标准库，无需单独安装
3. Numpy：实现数组，矩阵结构，支持面向线性代数的运算
4. Scipy：实现各种科学运算
5. Matplotlib：面向数据的绘图，绘制各种统计图
6. pillow：面向位图图像的处理，显示图像对图像做各种变形、变色等操作。
7. PyAudio：面向音频文档实施处理的库，对图像做各种变形，变色等操作。
8. jieba：面向中文的分词操作
9. Pandas：数据文件或数据库的访问库，包含面向多种储存结构的访问方法。
10. StatsModels：重要的数据分析工具，实现 T 检验、方差分析、回归分析等主要统计分析功能
11. requests：负责实现从客户端道 Web 服务器的连接，为获取 Web 信息提供网络爬虫所必须的支持
12. ScraPy：快速的、高层次的 Web 获取框架，提供了专业的网络爬虫功能，可实现 Web 数据的挖掘、分析、监控的功能
13. Beautifulsoup4：用于解析和处理 HTML 和 XML 格式的文档，帮助用户建立 HTML 文档的解析树
14. scikit-learn：面向机器学习的外部库，有限度地支持深度学习
15. theano：为执行深度学习中大规模神经网络算法的运算而设计的库，擅长处理多维数组，可高效运行于 CPU 和 GPU 上
16. MyQR：产生二维码

§ 6.2 在 Python 中使用外部库

6.2.1 安装

在命令行中输入：

```
1  pip install <外部库名称>
```
bush

若安装失败：

- 首先检查网络
- 检查外部库的版本号是否和 Python 匹配
- 如不匹配则需：

```
1  pip uninstall <外部库名称>
```

```
2  pip install <外部库名称> == 版本号
```

6.2.2 引用

1. `import <库名>`

   这种方式只是告诉当前程序即将使用此库，在使用库中的某个函数时，必须在函数名前添加库名作为标注。

```python
1  import math
2  math.sin()
```

2. `import <库名> as <库别名>`

   在使用库中的某个函数时，必须在函数名前添加库名作为标注。

```python
1  import math as mm
2  mm.sin()
```

3. `from <库名> import <类名>`

   可直接调用，但当引入大量库时，不建议这样操作。`from <库名> import *` 导入后所有成员，可无前缀直接调用，不用反复写库名。

§ 6.3 math 库及使用

math库中所有方法都是类方法，在调用时不需要创建math对象，而是以`math.方法名(参数)`的方式调用。

```python
1   impory math
2   math.e
3   math.pi
4   math.inf #无穷大
5   math.fabs(x) #绝对值
6   math.floor(x)
7   math.ceil()
8   math.turnc() #取整
9   math.exp()
10  math.log(double) #自然对数，其他情况使用换底公式
11  math.log2(double)
12  math.log10(double)
13  math.pow(x,y)
14  math.sqrt(x)
15  math.factorial(n) #阶乘
16  math.asin(x)
17  math.sin(x)
```

§ 6.4 turtle 库及使用

1. 关键语句

   (1) 移动类命令

```python
1  setup(width,height,startx,starty)
2  forward (长度x) / fd (长度x)
3  backword (长度x) / bk (长度x)
4  left(角度x)
5  right(角度x)
6  goto(x,y) / setxy(x,y)
```

   (2) 特殊图形

```python
1  circle(r,[弧度]) #绘制指定半径的圆，圆心默认在海龟左侧距离r的位置
```

```
clear()
begin_fill() #开始填充，在绘图之前调用
end_fill() #结束填充，在绘图之后调用
```

(3) 画笔情况

```python
pendown() / pd()
penup() / pu()
pensize(w) #设置画笔宽度，以像素为单位
pencolor(r,g,b) #RGB 0到1的浮点数
fillcolor(r,g,b)
speed(x) #0到10的整数
```

2. 示例

```python
from  turtle import*
penup() #抬起画笔
goto(20,0) #定位到合适位置
speed(10)
pencolor(0.92,0.92,0.01) #RGB 取0~1之间的浮点数
pendown() #落下画笔
fillcolor(0.92,0.4,0.4) #设置填充颜色为浅红色
begin_fill() #开始填充
for i in range(12):
    fd(165)
    left(75)
    fd(85)
    left(120)
goto(20,0) #绘制完成后回到起点，强制闭合图形，才能填充
end_fill()
done()
```

§ 6.5 random 库及使用

1. 关键语句

(1) 基础信息

```python
seed (x) #设置种子
```

(2) 基本随机数

```python
random() #返回(0.0,1.0)内的下一个随机浮点数
uniform(a,b) #返回一个取值在a,b之间的随机浮点数
randint() #返回[a,b]之间的随机整数
```

(3) 高级随机数

```python
randrange(start,stop,step) #用于在指定的整数序列中，随机返回一个整数，不会返回浮点数，只生成整数类型结果
normalvariate(x,y) #均值为x，标准差为y的正态分布序列，每次执行只获得一个数，多次操作可以获取多个数。
gauss(x,y)#同上，速度稍快
```

2. 实例

```python
import random
random.seed()
print ('使用默认的种子生成的随机数：',random.seed())
random.seed(10)
```

```
print ('使用种子10生成的随机数：')
ls1 = ls2 =[]
for i in range(100):
    ls1.append(random.gauss(5,1.43))
    #以高斯函数创建随机序列，每次一个数，通过列表ls1储存起来
print ("正态分布的伪序列：",ls1)
for i in range(50):
    ls2.append(random.normalvariate(5,1.43))
print ("正态分布的伪序列：",ls2)
```

第 7 章 文件管理与异常读取

§ 7.1 文件及其输入、输出

7.1.1 文件及其操作

1. 文件
   (1) 文件及其定义
      ① 处于活动状态（正在被应用或处理）的信息穿梭于计算机的内存和外存中，内存是其临时住所。
      ② 处于稳定状态且需要长久存储的信息都会以独立信息块的方式保存在计算机的外存中，这种存储于外存、带有用户指定的提取标志且包含若干二进制信息的信息块，就是计算机中的“文件”。
      ③ 计算机的文件可分为两类：
         • 直接面向应用、服务于用户的文档，称为数据文件。
         • 为了能够编辑、修改各种数据文件，满足各种用户需求而设计的应用程序
      ④ 在计算机语言中，通常使用 I/O 流的形式对文件进行读写操作。
   (2) 文件操作视角的类型划分
      ① 呈现文字材料；在 Python 中，主要是纯文本文档（*.txt 文档，简称 t 型文件）
      ② 带有复杂格式的多媒体文档或程序文档，他们主要是以二进制数据流的形式存在的，称为二进制文件（b 型文档），其操作是以字节为单位的。
2. 文件的操作
   (1) 文件操作主要有读和写两种，一方面是对源文件的读取操作，从 CPU 的视角看，是一种输入操作——数据从外存进入内存；一方面是对目标文件的写入操作，从 CPU 的视角看，是一种输出操作——数据从内存进入外存。
   (2) 二进制文档采用字节流的形式处理；文本类型的文件采用字符流的形式处理。

7.1.2 Python 中的文件操作

Python 中内置了 file 类，利用“open(文件名)”的方式获取 file 对象。

1. 文件操作命令
   (1) 文件的打开和关闭
      ① 打开文件的方法：文件变量 = open('文件名信息','打开方式')

```
t - 文本型文件
b - 二进制型文件
r - 只读取
w - 只写入
+ - 修改或新增
```

      ② 关闭文件的方法：文件变量.close()
   (2) 文件的读取操作

```Python
文件变量.read() #把读取的内容作为返回值
文件变量.read(x) #读取长度为x的内容
文件变量.readline() #直接从文件变量中读取一行内容，把读取的内容作为返回值
文件变量.readlines() #从文件变量中读取所有行，并把结果存入列表，每一行是列表的一个元素
```

(3) 文件的写出操作

```Python
文件变量.write(string) #把字符串变量的内容直接写到文件变量
文件变量.writeline(x) #把列表x中的内容逐行写到文件变量。列表的内容是直接写出的，不会自动加换行符。
```

(4) 文件的定位操作

```Python
文件变量.seek(lenth,pos) #以pos为起点，把文件指针移动到长度到len的位置。其中pos=0表示文件头，pos=1代表当前位置，pos=2代表文件尾
文件变量.next() #返回文件的下一行位置，并将文件指针移向下一行位置
文件变量.tell() #获取文件指针指向的当前位置
```

2. 示例

(1) 把内存信息写到文档

```Python
fw  = open('my.txt','wt+')
strs = input("请输入要存储的字符串")
while (strs != 'over'):
    fw.write(strs+'\n')
    strs = input("请输入要存储的字符串")
fw.flush() #强制把缓冲区所有内容写入硬盘，立刻生效
fw.seek(0) # 重新定位到文件开始处
for line in fw: #对文档中的每一行，做：
    print(line)
fw.close()
```

还可以使用 with 语句：

```Python
with open('my.txt','wt+', encoding='utf-8') as fw: # 加utf-8避免中文乱码
    strs = input("请输入要存储的字符串")
    while strs != 'over':
        fw.write(strs+'\n')
        strs = input("请输入要存储的字符串")
    fw.seek(0)
    for line in fw:
        print(line)
# 无需要写fw.close()、fw.flush()，with语句全部自动处理
```

(2) 文件复制功能的实现

```Python
fo = open('my.txt','rb') #打开源文件
fw = open('D:\\mynew.txt','wb') #打开新文件
fo.seek(0) #原文档定位到开头
for line in fo: #逐行读取并写盘
    fw.write(line)
fw.close()
fo.close()
print("数据复制完成了")
```

(3) 针对文档的加密

```Python
  def enCode (fo,fw,pswd):
    fo.seek(0)
```

```
    fd = fo.read()
    pw = len(pswd)

    i,j = 0,0
#i用来遍历「源文件二进制数据fd」的索引，指向当前正在处理的「文件字节」
#j 用来遍历「密码字符串pswd」的索引，指向当前正在使用的「密码字符」
    buff = []
    #创建空列表作为缓冲区，因为bytes是不可变类型，频繁拼接会反复创建新对象，效率极低；用列表append存数据，最后统一转换，是 Python 处理大量字节的最优写法。
    while(i<len(fd)):
        cc = fd[i]^ord(pswd[j])
        #ord()把单个字符转换成对应的 ASCII 码值
        j = (j+1) % pw
        #当j从 0 增加到 7 时，刚好用完 8 个密码字符；此时j+1=8，执行8%8=0，j会被重置为 0，重新从密码的第一个字符开始使用，循环往复。
        i += 1
        #将存储「加密后整数」的列表，转换为二进制字节串bytes类型。
        buff.append(cc)
    buff = bytes(buff)
    fw.write(buff)

def main():
    pswd = input("请输入8个英文字符作为密码")
    fo = open("source.txt",'rb')
    fw = open("screet.txt",'wb+')
    enCode(fo,fw,pswd)
    fo.close()
    fw.close()
    print("已完成加密")
main()
```

§ 7.2 异常处理

7.2.1 异常的概念和处理机制

1. 异常的概念

异常是由程序中的风险语句引起的，既不是语法错误，也不是逻辑错误，它是一种在语法和逻辑上都没有错误，但可能因运行环境的原因或用户操作不当而容易导致程序崩溃的语句。

2. 异常的处理机制

在一个函数运行的过程中，如果发生了异常，则此函数自动生成一个代表该异常的处理对象，并把此对象提交给“运行时”（run-time）系统来处理。运行时系统则根据接受到的代码来处理这一异常。

人们把创建对象并把它交给“运行时”的过程称为抛出（throw）异常，而“运行时”在系统运行栈中上溯并找到异常的真正原因、提出相应处理方法的过程，称为捕获（catch）并解决异常。

7.2.2 应用

1. 常见异常处理方法

(1) 异常处理基本结构

所有由系统定义的运行异常都可以由系统自动抛出，不需要提供专门的抛出异常语句。若某些语句有出现异常的可能，应该放置到具有“尝试”功能的语句块中。

```
try:                                                    Python
    带有风险的语句块
except[异常类型]:
```

```
4    处理异常现象的语句块
```

在特殊情况下，某个操作语句可能有多个异常，因此可能需要在多个 try 语句后附带多个 except 语句。

(2) 常见异常类型

```python
ImportError #模块导入异常
IOError #输入输出异常，文件I/O流传输出现问题时捕获的异常
KeyError #关键词异常，访问字典不存在的键
TypeError #数据类型异常
ZeroDivisionError #0作为除数的异常
OverflowError #数据溢出异常，在Python中无论是浮点数或是整数，他们能参与数学运算的长度都有限制
NameError #名称或标识符异常，访问当前程序中不存在的变量时引发的异常。
```

2. 示例

打开 myData.txt，然后把此文件赋值为名为 myNew.txt 的新文件。若文档中存在表达式，则计算表达式的值，并写入文档。

```python
try:
    fo = open("myData.txt",'rb')
    fw = open("myNew.txt",'rw')
    fo.seek(0)
    for line in fo:
        yy = eval(line)
        fw.write(line)
    fw.close()
    fo.close()
    print('复制数据已完成')
except IOError:
    print('文件打开或读写过程中出错了')
except ZeroDivisionError:
    print('除数不能为0')
```

第 8 章 科学计算：SciPy 和 NumPy

§ 8.1 NumPy 及其运用

8.1.1 数组与 NumPy

1. 数组

数组是由同类型的元素构成的有序序列。例如，10 个整数排序一个数列，可以用其下标标记每个整数的位置，这就是一个一维数组。

针对 C 语言等传统编程语言中数组定义的扩充——Python 提供了列表或元组，尽管扩大了数组对数据的容纳能力，却降低了运行效率。

array（数组）是 NumPy 中的主要数据类型，是面向同类型数据的序列。

2. NumPy 中的数组

(1) 创建数组

① 创建一维数组

```python
xx = np.array([1,2,3,4,5])
```

② 创建二维数组

```python
yy = np.array([[1,2,3],[4,5,6]])
```

(2) 对数组进行分片操作

```python
dd = yy[:2,:2] #从二维数组yy中抽取左上角的一小块（2*2）
```

(3) 针对数组的变形

对于二维数组，可以利用“reshape(行数，列数)”重构其行数、列数，形成新的二维数据，甚至把一维数据变形为二维数据。

```python
yy = xx.reshape(4,3)
```

reshape()是创建新的二维数组变量，结果存入新变量，原数组不变；resize()是原地修改。

(4) 对数组的转置

转置：行列互换，相当于数组被旋转 90°

```python
yy = xx.T
```

(5) 以 NumPy 自动生成数据

```python
xx = np.arange(起点,终点,步长)
xx = np.random.normal(均值,标准差,个数) #正态分布
```

3. 案例

(1) 定义数组

```python
import numpy as np
aa = np.array([1,2,3])
print("一维数组：\n",aa)
bb = np.array([[1,2,3],[4,5,6]])
print("二维数组：\n",bb)
cc = [1,2,3,4,5,6,7]
cc = np.array(cc)
print('list转数组：\n',cc)
dd = np.zeros([2,3])
print("创建值为0的二维数组：\n",dd)
ee = np.ones([3,2])
print("创建值为1的二维数组：\n",ee)

x = 8
ff = np.full((4,3),x)
print("创建值为x的二维数组：\n",ff)

gg = np.eye(4) #单位矩阵：对角线都是1，其余值为0
print("创建对角线是1，其余值为0的二维数组：\n",gg)
#显示数组的属性
print("数组的行数和列数",bb.shape)
print("数组的类型：",bb.dtype)
print("数组的维数：",bb.ndim)
print("数组的元素个数：",bb.size)
#对数组进行转置
print("对数组bb进行转置：\n",bb.T)

```

(2) 针对数组的切片

```python
import numpy as np
aa = np.array([[1,2,3],[4,5,6],[7,8,9]])
```

```
zz = np.array([np.arange(12)])
print(aa)
#定义一个复数形式的数组
yy = np.array([[1,2,3],[4,5,6],[7,8,9]],dtype=complex)
print("输出复数形式的数组的结果：",yy)
#基于数组的切片
bb = aa[0,2] #提取aa中0行2列的数，即3
print("输出单个元素：",bb)
bb = aa[[0,2,1],[1,2,0]] #提取aa中[0,1],[2,2],[1,0]的数
print("输出三个元素：",bb)
ti = np.array([0,2,1])
cc = aa[np.arange(3),ti]  #按照ti从aa中逐行提取数据,ti是列索引,np.arange(3)是行索引
print("逐行输出列号为(0,2,1)的元素：",cc)
```

(3) 基于数组的运算

```python
import numpy as np
aa = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12]])
zz = np.array([np.arange(12)])
print(zz)
zz = zz.reshape(3,4)
print(aa)
print(zz)
print("输出相加之后的结果：\n",aa+zz)
print("输出相减之后的结果：\n",aa-zz)
print("输出相乘之后的结果（对应位置乘法）：\n",aa*zz)
print("输出相除之后的结果（对应位置除法）：\n",zz/aa)

mm = [1,2,3,'Ma']
print('输出列表{0}乘3之后的效果：\n {1}'.format(mm,mm*3))
```

8.1.2 NumPy 中的矩阵

借助“np.matrix(二维数组类型)”或“np.asmatrix(二维数组类型)”即可把二维数组转化为矩阵，相应的运算即按矩阵的方式进行。

```python
import numpy as np
# 定义二维数组
aa = np.array([[1,2,3,4],[6,5,3,4],[29,28,27,22]])
zz = np.arange(12)
zzz = zz.reshape(3,4)    # 把zz转化为3行4列的数组
pp = np.matrix([1,2,3,4,5]) # 把数组转化为矩阵
mm = np.asmatrix(aa)
nn = np.asmatrix(zzz)

print("输出矩阵mm:\n",mm)
print("输出原矩阵:\n",pp)
print("输出原矩阵的转置矩阵:\n",mm.T)
print("输出矩阵mm的逆矩阵:\n",mm.I)
print("----------------------")
print("输出相加之后的结果:\n",mm+nn)
print("输出相减之后的结果:\n",mm-nn)
print("矩阵乘以固定值:\n",mm*3) # 注意矩阵相乘的原则,必须满足"行数列数=列行数"
xx = nn.reshape(4,3)
```

```
print("矩阵相乘之后的结果:\n",mm*xx)
print("--------------------求和操作--------------------")
print("输出原矩阵mm(列)和:\n",mm.sum(axis=0))
#axis=0按行操作，axis=1按列操作
print("输出原矩阵mm(行)和:\n",mm.sum(axis=1))
print("--------------------求最大值操作--------------------")
print("输出原矩阵mm的第1行所有列之最大值:\n",np.max(mm[1,:]))
print("输出原矩阵mm的第1行:\n",mm[1,:])
print("输出原矩阵mm的最大值:\n",np.max(mm))
print("输出原矩阵mm中所有列的最大值:\n",np.max(mm,0)) #省略了‘axis=’
print("输出原矩阵mm中所有行的最小值:\n",np.min(mm,1))
```

§ 8.2 SciPy 及其运用

1. SciPy 的常见功能

1	cluster	矢量运算、K 聚类类算法
2	constants	物理和数学上的一些常数
3	fftpack	快速傅里叶变换
4	integrate	积分和常微分方程的求解
5	interpolate	插值处理和平滑的样条函数
6	io	输入 / 输出
7	linalg	线性代数
8	ndimage	N 维图像的处理
9	ord	正交距离回归算法
10	optimize	做迭代及优化、方程寻根
11	signal	信号处理
12	sparse	稀疏矩阵
13	spatial	空间数据结构和算法
14	special	特殊函数
15	stats	统计分布和函数

2. 应用

(1) 导入

SciPy 依靠 NumPy 提供数据结构。

```python
import numpy as np
from scipy import 模块名
```

(2) 基于 SciPy 的复杂方程求解

使用 optimize 模块中的 fsolve()和 root()函数。

- 首先整理方程式，使得未知数均放在方程式的左边。
- 其次，用户自定义函数 func(x)描述函数式。
- 最后调用 SciPy 自带的 fsolve()和 root()功能，以 result = fsolve(func,常数项)或 result = root(func,常数项)描述所需的计算。

(3) 案例

① 一元简单方程式

```python
import numpy as np
from scipy import optimize as op
def func(x):
    return x**2 - 8*x + 15
result = op.fsolve(func,0)
print(result)
```

② 二元方程组

```python
import numpy as np
from scipy import optimize as op
def func(xx):
    x = xx[0]
    y = xx[1]
    return [x*x + 8*x + y*3,x*2 - x -21]
result = op.fsolve(func,[15,0])
print(result)
```

第 9 章 图像 & 图形：pillow 与 Matplotlib

§ 9.1 图形、图像的概念

1. 图形：指由点或线组成的画面，多数由绘图软件绘制。比较注重线条及其组合，**主要采用矢量形式存储。**
2. 图像：直接利用扫描仪扫描或数码相机对景物拍摄而成。比较注重区块和像素，**主要采用位图形式存储。**
3. 位图：把图像看作m行n列的矩形块，每一块叫做一个像素，分辨率就用$m \times n$表示。
4. 颜色模型：RGB 加色原理，用 #RRGGBB 方式呈现。

§ 9.2 基于 pillow 的图像处理

pillow 最常用的两个模块是 Image 和 ImageDraw。**在引用此库时标记是以前的 PIL。**

```python
img.format  #图像的格式，如 JPEG 等
img.size    #图像的大小、宽度、高度
img.mode    #图形的模式，如 RGB 图、CMYK 图
open (URL 或文件全称)  #打开指定的图像文件，本方法为类方法，以 Image 引领，获得 Image 型的对象 img。例如，img=Image.open ("E:/my.jpg")
img.show()  #显示此图片
img.save (文件全称) #把当前图像对象保存为新文件
img.resize (宽度，高度)  #文件进行缩放，缩放为新大小
img.rotate (角度) #按照角度进行*逆时针*旋转
img.filter (ImageFilter. 类型)
#对图像做各种特效操作，BLUR 代表模糊，DETAIL 代表细节增强，FIND_EDGES 代表寻找边缘，EMBOSS 代表浮雕效果，SHARPEN 代表锐化效果
img.thumbnail (宽度，高度) #对图像制作具备明确宽高的缩略图效果，原地修改!!!
resize (宽，高)，Image.NEAREST)  #绘制图像的翻转图
merge ("rgb", (红，绿，蓝))  #色彩合并，变色调整
```

示例：

```python
from PIL import Image
im = Image.open("D:\\Layout1.jpg")
print(im.mode,im.format,im.size)
im.show()
#缩放图片
height,width = 200,300
out = im.resize((width,height),Image.NEAREST)
out.show()
#旋转图片
our = im.rotate(90)
```

```
our.show()
#制作缩略图
im.thumbnail((128,128)) #以元组方式指定缩略图的大小
im.show
#变色与调整
r,g,b = im.split() #分解三基色
ourm = Image.merge('RGB',(b,g,r))
ourm.show()
#保存
im.save("D:\\image-2.png")
```

§ 9.3 基于 Matplotlib 制作统计图

9.3.1 Matplotlib 处理图像技术

1. 基本思路

(1) 获取画板对象 在任何绘图之前，均需要创建一个 Figure 对象，可以理解为需要一张画板才能开始绘图。

首先，“`import matplotlib.pyplot as plt;import numpy as np`”。

其次，获得画板对象。“`fig = plt.figure()`”

在画图完成后，务必记得显示图形：`plt.show()`

(2) 为 plt 配置图像的属性

```python
plt.grid() #网格线
plt.xlabel('字符串') #设置x轴的标题
plt.xlim(下限,上限) #设置x轴的上下限
plt.title('图表的标题')
plt.figtext(x,y,'标题文字',ha = 'center') #ha表示文字的水平对齐方式
```

2. 不同类型的统计图的绘制

(1) 绘制出折线图

```python
plt.plot(数据序列,"格式符",linewidth=线宽,label = '图例')
plt.plot(x序列,y序列) #以极简风格绘制折线图
```

在绘制图形过程中，若希望绘制相对平滑的曲线，可以利用 NumPy 中的线条平滑函数“`make_interp_spline(x,y)(x)`”对数据进行预处理，然后再绘制图像。

格式符可由颜色、点型、线型等构成，如“`go`”，“`rs`”等，分别代表绿色圆点圈，红色矩形块图。

(2) 绘制散点图

```python
plt.scatter(x数据序列,y数据序列,[其他参数])
```

(3) 饼图

```python
plt.pie(数据序列,[其他参数])
```

(4) 绘制直方图

```python
plt.hist(数据序列,[其他参数])
```

(5) 绘制箱体图

```python
plt.boxplot(数据序列,[其他参数])
```

第 10 章 文本处理与网络爬虫

§ 10.1 文本处理与可视化

10.1.1 中文文本处理入门

1. 中文分词原理 利用已经存在的中文分词库，将带分词的材料与中文分词库的内容进行对比，按照最大取词原则，通过图结构和动态取词方法，找到最大概率的词组。
2. 实现中文分词的方法
   (1) 常见分词方式

```python
jieba.lcut(myStr, cut_all=True)
#以全匹配模式进行分词（显示每一个组合）
jieba.lcut(myStr, cut_all=False)
#以精确匹配模式进行分词（显示最大词语组合）
jieba.lcut_for_search(mystr)  #搜索引擎模式，返回一个列表类型
jieba.analyse.extract_tags (content, topK = 数量)
#对 content 自动分词，并提取指定个数的关键词。此命令的使用面很广
jieba.analyse.extract_tags (content, topK = 数量, withWeight=True) # 在以指定关键词个数进行分词的同时，统计各词汇的权重
```

   (2) 示例：
   ① 利用 jieba 进行中文分词

```python
import jieba
mystr = "我来自北京师范大学！"
#采用全匹配模式进行中文分词
seg_list = jieba.lcut(mystr,cut_all = True)
print('全匹配分词：\n',seg_list)
#全匹配分词： ['我', '来自', '北京', '北京师范大学', '京师', '师范', '师范大学', '大学', '！']
#精准匹配模式，这是默认模式
seg_list = jieba.lcut(mystr,cut_all = False)
print('精准匹配分词：\n',seg_list)
#精准匹配分词：['我', '来自', '北京师范大学', '！']
#搜索引擎模式
seg_list = jieba.lcut_for_search(mystr)
print('搜索引擎分词：\n',seg_list)
#搜索引擎分词：['我', '来自', '北京', '京师', '师范', '大学', '北京师范大学', '！']
#向jieba库添加中文词
jieba.add_word("来自北京师范大学")
sg = jieba.lcut(mystr)
print('加入新词后的分词：\n',sg)
#加入新词后的分词：['我', '来自北京师范大学', '！']
#导入字典文件，是jieba分词库更丰富
jieba.load_userdict("D://mydictionary.txt")
```

   ② 利用 jieba 分析文本文件“西游记”

```python
import jieba.analyse
file = "D://my.txt"
with open(file, 'rb') as f:
    content_bytes = f.read()  # 二进制读取
# 解码：先尝试utf-8解码，失败则用gbk，再失败则忽略错误，转成字符串
```

```
try:
    content = content_bytes.decode('utf-8').strip()
except:
    try:
        content = content_bytes.decode('gbk').strip()
    except:
        content = content_bytes.decode('gbk', errors='ignore').strip()
#返回频数最高的20个关键词
topk = 20
tags = jieba.analyse.extract_tags(content,topK = topk)
print(tags)
print("="*30)
#withWeight = True 将权重值一起返回
tags = jieba.analyse.extract_tags(content,topK = topk,withWeight = True)
print(tags)
```

§ 10.2 以 WordCloud 绘制词云图

1. 步骤

(1) 使用 WordCloud 外部库

```
form worldcloud import WordCloud
```

(2) 读取数据文件

(3) 以 jieba 对数据文件做分词操作

首先，假设数据文件已经被读入 myContent 变量，用 jieba 对此变量进行分词操作。

```
myWord = jieba.lcut(myContent,cut_all = False)
```

然后，以空格符对分词结果做 join 串接。

```
myWord = " ".join(my_Word)
```

(4) 对词云图对象做简单配置：WordCloud(参数表)

常见配置项有：中文字库文件名、宽度、高度、词云图的背景等。

(5) 基于分词结果，生成词云图

```
result = WordCloud.generate(myWord)
result.to_file(存储文件名)
```

2. WordCloud 类中相关参数和方法

```
width / height  #指定生成词云图片的宽度、高度，若省略，默认为400像素
min_font_size / max_font_size #指定词云字体中的最小字号和最大字号，若省略，最小字号默认为4号
font_path #指定字体文件的所在路径，相当于指定所用的字体
font_step #指定词云字体字号之间的间隔，如果该参数不指定，则默认为 1
max_words #指定词云显示的最大单词数量，默认为 200 个词语
stop_words #指定词云的排除词列表，列入排除词列表的单词将不被显示在词云中
mask  #指定生成词云图的形状，如果需要非默认形状，则需要使用 imread () 函数引用其他图片
background_color  #指定词云图片的背景色，默认黑色
generate (txt 信息) #函数：针对指定的 TXT 信息执行生成词云的操作
to_file (词云图名)  #函数：保存词云图，形成词云文件
to_image()  #函数：直接转化为图像对象，以便按图像来操作
```

3. 示例：

```
import jieba.analyse as jb
from wordcloud import WordCloud
```

```
file = "D://my.txt"
content = open(file,'rb').read()
topK = 30
tags  = jb.extract_tags(content,topK=topK)
myWord = " ".join(tags)
stW = ['乃是','这般']
wc = WordCloud(font_path='C://Windows//Fonts//simsun.ttc',stopwords = stW,width = 600,height = 500)
wc.generate(myWord)
img = wc.to_image()
img.show()
file.close()
```

§ 10.3 Web 信息获取与爬虫入门

10.3.1 网络连接技术

1. 连接到 Web 网站

(1) 使用 requests 库

```
<连接变量名>=requests.get(网址,timeout='超时时间')
```

Python

此时可利用“<连接变量名>.raise_for_status()”获取连接状态。如果连接成功，则返回正确码 200。在实现正常连接之后，可用“<连接变量名>.encoding="语言编码"”语句设置当前 Web 的编码方式，常见 Web 编码方式有 GB/T 2312、UTF-8、GBK 等。

(2) 获取 Web 上的数据

连接成功后，通过“连接变量名”即可获取该 Web 页的全部数据。此时应使用命令：

```
myContent = <连接变量名>.text.
```

2. 示例：

```
import requests as rq
def getNetText(url):
    try:
        my = rq.get(url,timeout=30)
        my.raise_for_status()
        my.encoding='utf-8'
        return my.text
    except:
        return ''
def main():
    url = "https://www.baidu.com/"
    myTxt = getNetText(url)
    print(myTxt)
main()
```

Python

10.3.2 网络信息分解

1. 步骤

(1) 引用“form bs4 import BeautifulSoup”

(2) 对于获得的网页内容，须先用 BeautifulSoup 进行预处理，在处理时需要声明为“html.parser”类型。

```
sp = BeautifulSoup(my,'html.parser')
```

Python

(3) 利用循环语句重复执行指令“sp.find_all ("html 的特定标记符")”，以便获取该标记符下的属性值。

例如，要获取表格的值，可寻找“table”标记符；要获取表格中某行的值，可寻找“tr”标记符；要获取网页内各图片的信息，则需要获取“img”标记符的属性值。

(4) 每一步骤所获得的结果可用 append 命令添加到预定义的列表中。

(5) 输出经信息分解之后所获得的结果。

如果获得的对象仍是复杂结构，则需要进一步地细化检索其内部内容。另外，人们还经常使用“sp.prettify ()”命令预处理网页，以获得一个比较精简的网页数据信息，降低信息抓取的难度。

第 11 章 Python 环境下的统计分析

§ 11.1 统计推断的概念

1. 定义

(1) 基于原始数据的计算判断假设是否成立的分析方法。

2. 类别

(1) 差异显著性检验：t 检验、方差分析、基于交叉表的卡方检验

(2) 相关性分析：Pearson 相关性分析和 Spearman 相关性分析

检验名称	强度变量	检验概率	显著性标准
t 检验	T值	P(Sig)	$P < 0.05$
方差分析	F值	P(Sig)	$P < 0.05$
基于交叉表的卡方检验	卡方距离	P(Sig)	$P < 0.05$
相关性分析	相关系数r	P(Sig)	$P < 0.05$

3. 统计推断中的两类错误

在统计推断中，“显著性”是研究者期望所发生的事情。对于显著性的统计推断，容易发生两种错误：

其一，错误的拒绝了“0 假设”，把不该显著的事情判定为显著。（α错误，TYPE I 错误）；

其二，错误的接受了“0 假设”，把应该显著的事情判定为不显著。（β错误，TYPE II 错误）。

因此，人们常指定检验概率 Sig = 0.05 作为判断是否承认 0 假设的标准（即置信度为 95%）。当 Sig≥0.05 时，就承认 0 假设，认为不显著。Sig≤0.05 时，就拒绝 0 假设，认为显著。

§ 11.2 数据预处理

11.2.1 结构形式及处理

1. Python 支持的数据结构

数据集被严格地定义为二维表形式。在这个二维表中，每一行代表对一个研究对象的描述，称为个案或记录，每一列代表一个属性，每一列的数据均有相同的类型。**这种数据在 Python 中被定义为 DataFrame 类型。**

DataFrame 本质上是一种二维数组，但是比二维数组多了一个标题行，可以和二维数组灵活转换。

2. 不同结构的数据之间的转化

假设 df 为已经定义的 DataFrame，且已经 import panda as pd。

(1) list 转化为 array

```python
<新变量> = np.array(list名)
```

(2) 把新数据添加到二维列表内

```python
list名.append(其他一维列表)
```

(3) 把无标题的二维列表转化为 DataFrame 类型

```python
df = pd.DataFrame(list名)
df.columns = ['列名1','列名2'...]
```

(4) 针对 DataFrame 的操作

```python
dfNew = df[['列名1','列名2'...]]
#抽取某些列构成新DataFrame
```

(5) 把 DataFrame 转化为二维数组

```python
dArray = np.array(df)
```

(6) 两 DataFrame 的水平拼合（列合并）

```python
dfNew = pd.merge(df1,df2,how="left",on="合并关键词")
#需要公共变量作为基准，how说明以哪个表为基准，on说明连接关键词。
```

(7) 两 DataFrame 的垂直拼合（行合并）

```python
df = df.merge(df1)
#把DataFrame型的df1合并到df末尾，要求两表应有基本相同的列结构。
```

3. 示例：

```python
from pandas import *
import numpy as np
xx = np.array([[1,1,1,1,1],\
        [2,2,3,4,5],\
        [2,3,4,5,6],\
        [8,8,8,8,8]])
print('原始二维表：\n',xx)
df = DataFrame(xx) #把二维数组转化为DataFrame
print('new Dataframe:\n',df)
df.columns = ['A1','A2','A3','A4','A5']
print('添加列名后：\n',df)
df1 = df[['A1','A2','A3']]
print('截取部分列：\n',df1)
df = DataFrame.merge(df,df1,how='left',on='A1')
print('拼合df和df1:\n',df)
```

11.2.2 数据提取技术

1. 基于 Pandas 读取数据

(1) Pandas 的使用

① 打开 Excel 文档

```python
import pandas as pd
dat = pd.read_excel('D:\\mydata.xls','stu')
#打开mydata文档，读取stu工作表
#此时得到的dat为DataFrame格式的数据集，不是纯内容。
```

② 打开远程 MySQL 数据库，并读取其中特定数据表的内容

```python
import pymysql
import pandas as pd
#建立数据库连接
conn=pymysql.connect(host="172.22.126.61",\
    user="mxltext",password="m123@bnu",\
    database="myDB",port=3306,charset="utf8")
mydata = pd.read_sql('select * from student where 性别='男'',conn)
```

```
#执行数据库里的SQL查询语句，从数据表student里查询男生
#mydata里就是查到的数据，是DataFrame型数据
#在实际运用中应该替换成个人服务器的地址、账号和密码
```

③ 基于 xlrd 库读取 Excel 数据

```python
wb = xlrd.open_workbook('E:\\mydata.xls') #打开Excel文档
sheet = wb.sheet_by_name('stud')
#通过excel表格名称获取工作表，存入sheet表格中
#新变量sheet是一个二维数组
```

2. 示例：

```python
import xlrd

#定义新函数，把工作表内容按行读取并作为函数返回值
def excelRow():
    wb = xlrd.open_workbook('E:/myData2.xls')  # 打开Excel文件
    sheet = wb.sheet_by_name('stud')
        #通过excel表格名称(stud)获取工作表内容
    data = []                                  #创建空list
    for a in range(sheet.nrows):
        #循环读取表格内容（每次读取一行数据）
        row = sheet.row_values(a)              #每行数据赋值给row
        for xx in range(len(row)):
                dat=row[xx]
                #因为表内可能存在多列数据，0代表第一列数据，1代表第二列
                data.append(dat)
                #把每次循环读取的数据插入到list
    return data

#定义新函数，把工作表内容按行读取并作为函数返回值
def excelCol():
    wb = xlrd.open_workbook('E:/myData2.xls')  # 打开Excel文件
    sheet = wb.sheet_by_name('stud')
        #通过excel表格名称(stud)获取工作表的内容
    data = []                                  #创建空list
    for a in range(sheet.ncols):
        #循环读取表格内容（每次读取一列数据）
        col = sheet.col_values(a)              #每行数据赋值给col
        for xx in range(len(col)):
                dat=col[xx]
                #因为表内可能存在多行数据，0代表第一行数据，1代表第二行
                dat.append(dat)
                #把每次循环读取的数据插入到list
    return dat

myDataC = excelCol()       #按列读取的方式返回工作表的内容（更常用）
print(myDataC)

myDataR = excelRow()       #按行读取的方式返回工作表的内容
print(myDataC)
```

11.2.3 面向数据的统计描述

1. 基于数据集的统计描述

(1) 求取描述统计量 所谓描述统计量，指类似均值、方差、标准差等的描述型变量。

① 求取均值 <数组变量名>.mean()

② 求取标准差 <数组变量名>.std()

(2) 绘制箱体图

```python
plt.boxplot(x=数组名, whis = 1.5, widths = 0.7, boxprops ={'color':'red'})
```

(3) 以 K-S 算法做正态性判定

若$P < 0.05$,则表示不为正态；若$P > 0.05$则表示为正态

```python
import scipy.stats as sas
res = sas.kstest(rvs = 数组名,args=(数组名.mean(),数组名.std()),cdf='norm')
```

(4) 以 S-W 算法做正态性判定

若$P < 0.05$,则表示不为正态；若$P > 0.05$则表示为正态。适用于小规模的样本数据，通常在 500 以内。

```python
import scipy.stats as sas
res = sas.shapiro(data)
```

§ 11.3 数据统计分析技术

11.3.1 相关性分析

1. 概念

相关性分析常用系数r来描述。**一般地，若 $|r| \geqslant 0.8$，则认为两列数据高度相关；若$0.4 \leqslant |r| < 0.8$，则认为两列数据相关；若$|r| > 0.4$，则认为两列数据不相关。$r > 0$，则称两组数据正相关，反之则为负相关。**

在 Python 中，实现相关性分析的操作语句如下，其中，res 中存储的是相关系数r。

```python
import numpy as np
res = np.corrcoef(配队列1,配队列2)
```

另外，若人们拿到的数据为行列规范的二维数组 XX，那么执行相关性分析还可以使用命令“res=np.corrcoef(XX, rowvar=False)”，以分析各列数据之间的相关性，其结果是一个行列数相等的对称矩阵。其中，res[0,1]代表了第 0 列与第 1 列之间的相关系数，res[0,2]则代表第 0 列与第 2 列之间的相关系数。而命令“res=np.corrcoef(XX, rowvar=True)”则可以分析 XX 中各行数据之间的相关性，获得相关系数r。

2. 示例：

```python
import pandas as pd
import numpy as np

x1=np.array([1,2,3,4,9])
x2=np.array([3,2,5,4,52])

print(x1)
print(x2)

res=np.corrcoef(x1,x2)
print(res)

print("==========================")
```

```
xx=np.array([[1,2,3,4,9],[3,2,5,4,52],[3,1,5,7,45]])
rs=np.corrcoef(xx)
print(rs)

rs2=np.corrcoef(xx,rowvar=False)    #列做相关性分析
print(xx)
print(rs2)

rs2=np.corrcoef(xx,rowvar=False)    #行做相关性分析
print(xx)
print(rs2)

print("=========================\n面向数据文件的列相关性")
dat=pd.read_excel("E:/myData.xls","stud")
df=dat[["Chinese","FengGe","Maths","Physics"]]
xx=np.array(df)
rs3=np.corrcoef(xx,rowvar=False)    #行做相关性分析
print(rs3)
```

11.3.2 差异性分析——t 检验

1. 概念

(1) 两配对样本 t 检验

针对两个满足配对条件（个数相同且具有一一对应关系）的样本，通过 t 检验分析其是否存在者显著性差异。**两配对样本 t 检验主要用于检查同一组测试被试两次测量结果之间的差异程度，常常用于检查测试样本在受到某种干预前后是否有显著的变化。**

先假设两列数据之间**不存在显著差异**，然后对这两列数据实施 t 检验，用函数的运算结果表示假设成立的概率。若函数运算结果（P）小于 0.05，这表示这两列数据**没有差别**的可能性低于 5%，说明两列数据是**具有显著差异**的。

在 Python 中，实现两配对样本 t 检验的操作语句如下，其中，res 是一个字典变量，存储了反映差异水平的变量T值和无显著差别的概率值P。

```python
import scipy.stasts as sas
res = sas.ttest_rel(配队列1,配队列2)
#以res中pvalue<0.05代表存在显著差异。
```

(2) 独立样本 t 检验

如果两独立样本数据具有相同的值域，而且基本满足正态分布，则可以借助统计推断中的 t 检验判断是否存在显著性差异。

首先需检查两列数据的方差是否齐性，如果方差齐性（即两组数据的波动程度相差不大），则使用**独立样本等方差 t 检验**，否则使用**独立样本异方差 t 检验**。

在 Python 中，实现独立样本 t 检验的操作语句如下，其中，res 是一个字典变量，存储了反映差异水平的变量T值和无显著差别的概率值P。

```python
import scipy.stasts as sas
res = sas.ttest_ind(配队列1,配队列2,equal_var=False)
#齐性为True，非齐性为False
#以res中pvalue<0.05代表存在显著差异。
```

方差齐性的检验如下：

```python
import scipy.stats as sas
stdOK = sas.levene(dat1,dat2)
```

11.3.3 回归分析和回归模型

1. 回归分析的质量评价

(1) 判定系数R^2：

$$R^2 = \frac{\text{回归值的平方和}}{\text{回归值的平方和+残差的平方和}}$$

反应的是回归方程和观测值的拟合程度，在一般应用中R^2最好大于0.6。在特殊情况下（如医疗领域），大于 0.3 也能被认为是有价值的方程。

(2) 回归模型的F值：

$$F = \frac{\text{回归均方和}}{\text{残差均方和}}$$

F值应该在 3.84 以上，否则可被认为无效。

(3) 回归模型的概率检验值（Sig 值）：

该值反映了整个回归方程的影响力。与 t 检验的思路一致，在置信度 95% 的条件下，当 Sig <0.05 时候，就可以认为该模型的影响力是显著的。

2. Python 对回归分析的支持

对于回归分析，通常使用 DataFrame 类型数据。

(1) 一元线性回归分析

```python
import statsmodels.api as sm
res = sm.OLS(y,x) #x为自变量序列，y为因变量序列
res = res.fit() #调整模型状态
return res.summary()  #输出模型结果
```

(2) 多元线性回归分析

- 方法一：

```python
import statsmodels.api as sm
res = sm.OLS(y,df) #因变量单独写，以dataframe型自变量集做自变量
res = res.fit()
return res.summary()
```

- 方法二：

```python
from statsmodels.fomula.api import OLS
res = ols('因变量名~自变量1+自变量2+自变量3',data=数据集合)
res = res.fit()
return res.summary()
```

注意：回归模型中，以R^2较大，自变量个数较少为佳。

11.3.4 归因分析——方差分析

对于多独立样本之间的差异性检验，需要使用方差分析。常用的方差分析有单因素方差分析和多因素方差分析两种方法。在 Python 中，stats 模块把方差分析与回归分析糅合在一起，实现单因素 / 多因素方差分析的命令块如下：

```python
from statsmodels.formula.api import ols
import statsmodels.api as sm
res=ols("因变量~分组变量",data=df).fit()
res=sm.stats.anova_lm(res,type=2)
```

在执行方差分析前，需要事先把数据整理到 DataFrame 中，然后调用回归分析的指令 OLS 做一次回归分析，探究因变量与分组变量之间的关系，最后执行方差分析的指令获取方差分析的结果。与 t 检验的思路相同，若检验概率$P < 0.05$，则认为分组之间存在显著性差异；否则，认为分组之间不存在显著差异。

从方差分析的定义看，方差分析探索的是：在以某个因素作为分组标准对样本分组后，检验各分组在因变量的取值上是否存在显著的差别。比如，以学生身高作为因变量，以学生籍贯作为因素变量，检验籍贯不同的学生在身高方面是否存在着显著的差别。

尽管从本质上看，方差分析检验的是分组之间的差异性，但在实际应用中，这种差异性检验又可以转换为对两变量之间的关联性分析。正如前例所讨论的，若不同籍贯的学生在身高方面存在显著性差异，我们就可以认为“身高”与“籍贯”存在着一定的关联性。进而从研究问题的逻辑起点出发，可以认为学生的身高受到了其籍贯的影响，即籍贯是身高的影响因素——因此，方差分析在很大程度被作为“归因分析”的重要研究方法。

对于值域“一低一高”型的两配对样本，人们经常以低值域变量作为分组基准，借助方差分析检验不同分组在高值域变量上的差异是否显著。进而通过差异的显著性水平，探究低值域变量能否直接影响到高值域变量的取值，分析低值域变量是否为高值域变量的重要影响因素。